SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Name	Chennuri Sumedha
Register Number	1971260029
Course Code & Name	C Programming
Slot	C
Assignment / Task Title	Structures, Unions and Storage Classes (Tasks 1 to 6) _
GitHub Repository URL	https://github.com/sumedha-2805/CSA0210.git
Submission Date	13-11-2026

Instructions to Students

1. Complete every sectionfor eachassigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — DATA DESIGN REVIEW (Patient Register)

Task / Problem Statement

Ajunior developer storedthe patient register of a primary health centre in five separate arrays (name, age, phone, bp, visitDate) with a global count and a function addPatient() that takes five separate values. Find the design problems, redesign the data with structures (nested structures for date and blood pressure) and rewrite addPatient() to accept a structure pointer.

Aim

Tofindthe weak points of a poor data design and repair it using structures, nested structures and a structure pointer.

Concept / Theory

A**structure** groups different data types in one unit. A **nested structure** is a structure used as a member of another structure (the date and the BP are inside the patient). A structure variable is accessed with the dot operator, and a structure pointer is accessed with the arrow operator (p->age).

Parallel arrays keep related data in different arrays that are connected only by the index, which is risky. An **array of structures** keeps each record together.

Procedure / Algorithm

1. Study theold code and list the design problems.
2. Define struct Date and struct BP, then struct Patient that contains them.
3. Write addPatient() that takes a pointer to a patient, checks the limit and copies the record.
4. Write sortByAge() (selection sort that swaps whole structures) and show().
5. Add 3 patients, print before and after sorting, then print sizeof(struct Patient).

Program / Source Code

```
#include <stdio.h>
#include <string.h>


#define MAX 100

struct Date { int day, month, year; };
struct BP   { int systolic, diastolic; };

struct Patient {
    char name[20];
    int age;
    char phone[11];          /* 10 digits + '\0' */
    struct BP bp;
    struct Date visit;
};

/* Adds one patient through a structure pointer; returns 1 on success */ int
addPatient(struct Patient list[], int *count, const struct Patient *p) {

    if (*count >= MAX) {
        printf("Register full!\n");
        return 0;
    }
    list[*count] = *p;
}   (*count)++;
    return 1;
/* Sort by age (selection sort): one whole structure is swapped */
void sortByAge(struct Patient list[], int n)
{


    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (list[j].age < list[min].age) min = j;
        if (min != i) {
            struct Patient t = list[i];
            list[i] = list[min];
            list[min] = t;
```

```
            }
        }
}

void show(const struct Patient *p) {

    printf("%-8s %2d      %s   BP %d/%d   %02d/%02d/%d\n", p->name, p->age, p->phone,
}           p->bp.systolic, p->bp.diastolic,
            p->visit.day, p->visit.month, p->visit.year);
int main(void)
{


    struct Patient list[MAX];
    int count = 0;

    struct Patient a = {"Ravi", 45, "9876543210", {120, 80}, {10, 11, 2026}};
    struct Patient b = {"Anita", 28, "9123456780", {110, 70}, {11, 11, 2026}};
    struct Patient c = {"Kumar", 62, "9988776655", {150, 95}, {12, 11, 2026}};
    addPatient(list, &count, &a);
    addPatient(list, &count, &b);
    addPatient(list, &count, &c);

    printf("Before sorting:\n");
    for (int i = 0; i < count; i++) show(&list[i]);
}
    sortByAge(list, count);
    printf("After sorting by age:\n");
    for (int i = 0; i < count; i++) show(&list[i]);

    printf("sizeof(struct Patient) = %zu bytes\n", sizeof(struct Patient));
    return 0;
```

Output Screenshot

```
$ gcc -Wall -Wextra t1.c -o t1
$ ./t1
Before sorting:
Ravi     45   9876543210 BP  120/80   10/11/2026
Anita    28   9123456780 BP  110/70   11/11/2026
Kumar    62 9988776655    BP 150/95    12/11/2026
After sorting by age:
Anita    28   9123456780 BP  110/70   11/11/2026
Ravi     45   9876543210 BP  120/80   10/11/2026
Kumar    62 9988776655    BP 150/95    12/11/2026
sizeof(struct Patient) = 56 bytes
```

Analysis / Discussion

Task 1.1 – Diagnose the Design (deficiencies and what could go wrong)

Deficiency	What could go wrong in the health centre
1. Parallel arrays (name, age, phone, bp, visitDate)	The five arrays are linked only by the index number. If one array is updated and another is forgotten, a patient's name can end up with another patient's age or phone number. A wrong record could be given to the doctor.
2. Phone number in char[10]	10 digits need 11 bytes because of the '\0' at the end. strcpy writes one byte too many and overwrites the next patient's phone number, so numbers get joined or damaged and the patient cannot be called.
3. Blood pressure (two values) in one float	120/80 has two numbers (systolic and diastolic). One float cannot keep both, so one value is lost. A high-BP patient may be shown as normal.
4. Date as text of 8 characters	"13/11/26" is 8 characters and needs a 9th byte for '\0', so it overflows. Text dates also cannot be compared or subtracted, so 'days since last visit' is hard to find.
5. No limit check on count	The 101st patient is written outside the arrays. The program can crash or damage other data, and registrations are lost.
6. Global variables	Any function can change the arrays or count by mistake, and bugs are hard to trace. The same code cannot manage two registers (for example two clinics).
7. Difficult to sort by age	To sort by age, all five arrays must be swapped together in every step. Missing one swap mixes up patient data.

Task 1.2 – Repair the Design

The data is now one **struct Patient** with nested structures **struct BP** and **struct Date**. phone[11] has space for '\0'. addPatient() takes a structure pointer, checks the limit and keeps count in the caller (no globals). The full code and output are shown above.

Task 1.3 – Design Defence (under 100 words)

In the structure design, all details of a patient stay together in one record. To sort by age we swap one whole structure, so a name can never be separated from its age. A patient can be passed to a function with one pointer instead of five values. New data, like a blood group, is added as one new member and the old code still works. Nested structures keep date and blood pressure clear, and the date can be compared easily. A count check stops overflow.

Alternative Concept and Complexity Analysis

When are parallel arrays still better? When a program works on only one field for all patients, for example finding the average age. The ages are stored together in memory, so scanning is fast and no padding is wasted. Parallel arrays are also good when a field is added only for a few operations.

Time to sort n patients (selection sort): both designs need about n(n-1)/2 comparisons, which is $O(n^2)$, and at most n-1 swaps. The difference is the data moved in each swap. With the structure array one swap copies the whole structure (sizeof(struct Patient) = 56 bytes, copied 3 times because of the temporary). With parallel arrays one swap needs five separate swaps (20+4+10+4+8 = 46 bytes), and it is easy to forget one.

Space: parallel arrays use 46 bytes per patient = 4,600 bytes for 100 patients, with no padding. The structure array uses 56 bytes per patient = 5,600 bytes. The extra bytes are the 1 extra byte for '\0' in phone and 1 padding byte (the structure's size is a multiple of 4 because of alignment). The small extra space is a good price for safer code.

Result

The old design was diagnosed (7 problems explained) and repaired with nested structures and a structure pointer. The program works: the patients were sorted by age correctly (28, 45, 62). I learned that keeping related data together in a structure makes sorting and passing to functions much safer.

TASK 2 — FROM REQUIREMENT TO NESTED STRUCTURES (Courier Parcel Tracking)

Task / Problem Statement

A courier franchise books parcels at its counter. Each parcel has an ID, weight, booking date, sender and receiver (each with name, city and PIN code), the service type (normal or express) and the charge. The franchise needs a list of parcels sorted by weight and a list of parcels for a given city. Identify the entities, define the structures, write the pseudocode and access expressions, and complete the given program (print_parcel and sort_by_weight).

Aim

Toconvert a real requirement into nested structures and an array of structures, and to sort structures by one member.

Concept / Theory

Nestedstructures are used when one entity contains another (a parcel contains a date and addresses). With an array of structures, parcel[i].receiver.city reaches an inner member by the dot operator. With a structure pointer, ptr->receiver.city does the same.

Selection sort finds the smallest weight in the remaining part and swaps it to position i. The swap copies the whole structure, so the id, address and date always travel together with the weight.

Procedure / Algorithm

1. Find the entities(Date,Person, Parcel) and decide which are nested.
2. Write the structure definitions and an array for the day's bookings.
3. Write print_parcel() using the arrow operator and sort_by_weight() using selection sort.
4. In main(), store 3 parcels, sort them, print them and show both access expressions.
5. Print sizeof(struct parcel), which is the number of bytes copied in each swap.

Program / Source Code

Part A – Structure definitions for the full requirement (Task 2.2)

```
/* Task 2.2 - structure definitions for the full requirement */
typedef struct { int day, month, year; } Date;

typedef struct {
      char   name[20];
      char city[20]; int
      pincode;
} Person;                                   /* used for both sender and receiver */

typedef enum { NORMAL, EXPRESS } Service;

typedef struct {
      int         id;
      float       weight;
      Date        booked;                   /*  nested  structure  */  /*
      Person sender;                        nested   structure   */   /*
      Person receiver;                      nested structure */
      Service service;
      float       charge;
} Parcel;

Parcel   bookings[100];   int               /* array of structures: the day's bookings */
main(void)
{

      Parcel *ptr = bookings;
}     bookings[2].receiver.pincode = 625001;          /* (i)       array + dot operator          */
      return (ptr + 2)->receiver.pincode == 625001 ? 0 : 1;                 /* (ii) pointer + arrow */
```

Part B – Program (TODO 1 and TODO 2 completed)

```
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
```

```
struct parcel { int id; float weight; struct address to; struct date booked; };

/* flat version, only for the size comparison */
struct flat_parcel { int id; float weight; char city[20]; int pincode;
                     int day, month, year; };

void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%d\n", p->id, p->weight, p->to.city,
}           p->to.pincode, p->booked.day, p->booked.month, p->booked.year);

/* Selection sort of the parcels in ascending order of weight. */ void
sort_by_weight(struct parcel p[], int n)
{

    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) {
            struct parcel t = p[i];
            p[i] = p[min];
            p[min] = t;
}       }
    }
int main(void)
{


    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}}, {502,
        1.2f, {"Chennai", 600077}, {12, 10, 2026}}, {503, 2.8f,
        {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);

    struct parcel *ptr = list;
    printf("Third parcel PIN (dot)          : %d\n", list[2].to.pincode);
}   printf("Third parcel PIN (arrow): %d\n", (ptr + 2)->to.pincode);
    printf("Bytes copied per swap (sizeof(struct parcel)) = %zu\n", sizeof(struct parcel));
    printf("sizeof nested = %zu, sizeof flat = %zu\n", sizeof(struct parcel), sizeof(struct flat_parcel));
    return 0;
```

Output Screenshot

```
$ gcc -Wall -Wextra t2.c -o t2
$ ./t2
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
Third parcel PIN (dot)          : 625001
Third parcel PIN (arrow): 625001
Bytes copied per swap (sizeof(struct parcel)) = 44
sizeof nested = 44, sizeof flat = 44
$ gcc t2_defs.c -o t2d && ./t2d && echo definitions OK
definitions OK
```

Analysis / Discussion

Task 2.1 – Identify the Entities

Structure	Fields	Nested inside
Date	day, month, year	used inside Parcel (booking date)
Person	name, city, pincode	used twice inside Parcel (sender and receiver)
Parcel	id, weight, booked (Date), sender (Person), receiver (Person), service (normal/express), charge	contains Date and Person

Task 2.3 – Design the Operations (pseudocode)

```
(a) Sort parcels by weight (selection sort)
    for i = 0 to n-2
        min = i
```

```
        for j = i+1 to n-1
            if parcel[j].weight < parcel[min].weight then min = j
        if min != i then swap the whole structures parcel[i] and parcel[min]
```

```
(b) List all parcels for a given receiver city
    read city
    found = 0
    for i = 0 to n-1
        if parcel[i].receiver.city equals city then print parcel[i]; found = 1
    if found == 0 then print "No parcels for this city"
```

Task 2.4 – Access Paths (PIN code of the receiver of the third parcel)

(i) Array and dot operator: **bookings[2].receiver.pincode**

(ii) Structure pointer and arrow operator, with Parcel *ptr = bookings: **(ptr + 2)->receiver.pincode** (also written ptr[2].receiver.pincode)

Alternative Concept and Complexity Analysis (flat structure)

If the address and date were written directly inside the parcel (a flat structure), the fields would have to be repeated: sender_city, sender_pincode, receiver_city, receiver_pincode, day, month, year. The access path becomes shorter (p.receiver_pincode instead of p.receiver.pincode), but the address type cannot be reused, so one function like print_address(const struct address *a) cannot work for both sender and receiver, and the date cannot be passed alone to a date function.

Space (measured with sizeof): nested = 44 bytes and flat = 44 bytes for the same fields (4+4+20+4+12). There is no padding in this case, so the nested design costs no extra memory and is better for reuse. **Time:** sorting is $O(n^2)$ comparisons in both designs. Each swap copies sizeof(struct parcel) = 44 bytes in both. The nested access has no extra cost, because the offsets are fixed at compile time.

Result

The program works and prints the parcels in ascending order of weight (502, 503, 501), exactly as expected. Each swap copies 44 bytes. The nested design is easy to read and reuse, and it uses the same memory as the flat design.

TASK 3 — UNIONS AND MEMORY LAYOUT (Weather-Station Sensor Packets)

Task / Problem Statement

A weather station sends packets that carry exactly one kind of value: a temperature (float), a humidity percentage (int) or a short status text (up to 7 characters). The station has very little memory and stores thousands of packets. Design the packet as a structure and as a tagged union, predict and verify sizeof, implement the tagged union with a printing function, and test what happens when the wrong member is read.

Aim

Tocompare a structure and a tagged union for memory use, and to understand size, alignment and the misuse of a union.

Concept / Theory

A **union** keeps all its members at the same address, so its size is the size of the biggest member. Only one member can be valid at a time. A **tagged union** is a structure with an extra field (an enum tag) that remembers which member is valid.

Alignment and padding: the compiler places each member at an address that suits its type, and makes the total size a multiple of the largest alignment. Extra bytes added for this are called padding.

Procedure / Algorithm

1. Define theenum kind (TEMPERATURE, HUMIDITY, STATUS) and the tagged union struct reading.
2. Write show(), a switch on r->type that prints only the valid member (TODO 1).
3. In main(), create 3 readings (one of each kind) and print sizeof for the union, the tagged structure and the plain structure.
4. Call show() for every reading.
5. Store a temperature, read the humidity member, and record the output.

Program / Source Code

```
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

/* Design A: plain structure holding all three fields */
struct packet_struct { float celsius; int percent; char status[8]; };
/* Design B: tagged union */
struct reading {

      enum kind type;
      union { float celsius; int percent; char status[8]; } value;
};

void show(const struct reading *r) {

      switch (r->type) {
            case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
            case HUMIDITY:        printf("Humidity %d %%\n", r->value.percent); break;
}           case STATUS:          printf("Status %s\n", r->value.status); break;
      }
int main(void)
{


      struct reading log[3] = {
            {TEMPERATURE, {.celsius = 31.5f}},
            {HUMIDITY,       {.percent   =   68}},
            {STATUS,         {.status = "OK"}}
      };
      printf("sizeof union = %zu, sizeof struct reading = %zu\n",
                  sizeof log[0].value, sizeof log[0]);
      printf("sizeof struct packet_struct = %zu\n", sizeof(struct packet_struct));
      for (int i = 0; i < 3; i++) show(&log[i]);

      /* Misuse experiment: store temperature, read humidity */
```

```
    struct reading m;
    m.type = TEMPERATURE;
    m.value.celsius = 31.5f;
    printf("Misuse: stored 31.5, read percent = %d\n", m.value.percent);
    return 0;
}
```

Output Screenshot

```
$ gcc -Wall -Wextra t3.c -o t3
$ ./t3
sizeof union = 8, sizeof struct reading = 12
sizeof struct packet_struct = 16
Temp 31.5 C
Humidity 68 %
Status OK
Misuse: stored 31.5, read percent = 1107034112
```

Analysis / Discussion

Task 3.1 – Structure or Union?

Design A (structure): struct packet_struct { float celsius; int percent; char status[8]; } holds all three values at the same time. **Design B (tagged union):** struct reading { enum kind type; union { float celsius; int percent; char status[8]; } value; }. A packet carries exactly one kind of value, so the union shares one memory area, and the tag (type) tells which member is valid. **Design B suits the station**, because it has very little memory and stores thousands of packets: it saves 4 bytes in every packet.

Task 3.2 – Predict the Size (64-bit system) and verify

Type	Prediction and reason	Measured
struct packet_struct	float 4 + int 4 + char[8] 8 = 16. The largest alignment is 4, and 16 is a multiple of 4, so there is no padding.	16
union (value)	The size of a union is the size of its largest member = char[8] = 8 (float and int need 4). 8 is a multiple of 4, so no padding.	8
struct reading (tag + union)	enum 4 + union 8 = 12, alignment 4, so no padding.	12

The structure is **not** 4 + 4 + 4 + 8 = 20 bytes, because the union members share the same memory. The union takes only 8 bytes (the biggest member), not 4 + 4 + 8. So the structure is 4 (tag) + 8 (union) = 12 bytes.

Task 3.4 – Misuse Experiment

A temperature of 31.5 was stored and then the humidity (int) member was read. The output was **1107034112**, not 31. A float is saved in memory as IEEE-754 bits; 31.5 is stored as the bit pattern 0x41FC0000. Reading the same four bytes as an int gives the number 0x41FC0000 = 1107034112. The union does not convert anything. That is why the tag must be checked before reading a member.

Alternative Concept and Complexity Analysis

Three separate arrays (one per type): each packet needs only its own size (4, 4 or 8 bytes), but the program must also keep the order of arrival (an extra index or tag per packet), so the saving is lost, and reading a packet in arrival order is harder. **malloc with a void pointer per packet:** each packet needs an 8-byte pointer, plus the tag, plus the memory manager's own bookkeeping (usually 16 to 32 bytes per allocation), so it is much bigger than 12 bytes. It is also slower, because the pointer has to be followed, and memory can be fragmented. **Tagged union:** reading a packet is O(1) (check the tag, read one member) and the cost is a fixed 12 bytes per packet, in one continuous array, so it is the best design here.

Result

The tagged union works and prints each packet according to its tag. The measured sizes (union 8, struct reading 12, plain structure 16) match the predictions. The misuse experiment gave 1107034112, which proved that a union gives no safety by itself and the tag must always be checked.

TASK 4 — STRUCTURE FORENSICS (Savings-Account Interest)

Task / Problem Statement

Abank'ssavings-accountprogram is supposed to add interest to every account. After running it, every balance is unchanged, and one statement does not compile. Find the two defects, prove the first one by printing the address of the structure inside addInterest() and in main(), repair the program using a structure pointer and the arrow operator, and explain when a structure should be passed by value, by pointer and by const pointer.

Aim

To find why structure changes are lost when a structure is passed by value, and to repair the program using a pointer and the arrow operator.

Concept / Theory

Pass by value: the function gets a copy, so changes are lost. **Pass by pointer:** the function gets the address of the original, so changes are kept. The arrow operator (a->balance) is the short form of (*a).balance.

The dot operator is used with a structure variable, and the arrow operator with a pointer to a structure. Using the wrong one is a compile-time error.

Procedure / Algorithm

1. Read the program and find why balances stay the same and why one line fails to compile.
2. Print the address of the structure inside addInterest() and the address of list[0] in main().
3. Change the parameter to struct account * and use the arrow operator inside the function.
4. Call addInterest(&list;[i], 4) for each account and print every balance.
5. Compare with the pass-by-value-and-return version.

Program / Source Code

Original program (as given, with the two defects)

```
#include <stdio.h>
struct account { int number; float balance; };

void addInterest(struct account a, float rate) {
      a.balance = a.balance + a.balance * rate / 100;
}

int main(void) {
      struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
      struct account *p = list;
      for (int i = 0; i < 3; i++) addInterest(list[i], 4);
      printf("%d %f\n", p.number, p.balance);
      return 0;
}
```

Evidence program (prints the address inside addInterest() and in main())

```
#include <stdio.h>
struct account { int number; float balance; };

void addInterest(struct account a, float rate) {
      printf("inside addInterest : address of a                    = %p\n", (void *)&a);
      a.balance = a.balance + a.balance * rate / 100;
}

int main(void) {
      struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
      struct account *p = list;
      printf("inside main              : address of list[0] = %p\n", (void *)&list[0]);
      for (int i = 0; i < 3; i++) addInterest(list[i], 4);
      for (int i = 0; i < 3; i++) printf("%d %.2f\n", (p + i)->number, (p + i)->balance);
      return 0;
}
```

Repaired program (structure pointer and arrow operator)

```
#include <stdio.h>
struct account { int number; float balance; };
```

```c
void addInterest(struct account *a, float rate) {
    printf("inside addInterest : address of account = %p\n", (void *)a);
    a->balance = a->balance + a->balance * rate / 100;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    printf("inside main                : address of list[0] = %p\n", (void *)&list[0]);
    for (int i = 0; i < 3; i++) addInterest(&list[i], 4);
    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", list[i].number, list[i].balance);
    return 0;
}
```

Output Screenshot

```
$ gcc t4_orig.c -o t4o
t4_orig.c: In function ‘main’:
t4_orig.c:12:24: error: ‘p’ is a pointer; did you mean to use ‘->’?
       12 |       printf("%d %f\n", p.number, p.balance);
          |       ^
          |       ->
t4_orig.c:12:34: error: ‘p’ is a pointer; did you mean to use ‘->’?
       12 |       printf("%d %f\n", p.number, p.balance);
          |                                  ^
          |                                 ->
$ gcc t4_evidence.c -o t4e && ./t4e
inside main                : address of list[0] = 0x7fff650d8900
inside addInterest : address of a                = 0x7fff650d88d8
inside addInterest : address of a                = 0x7fff650d88d8
inside addInterest : address of a                = 0x7fff650d88d8
101 5000.00
102 12000.00
103 800.00
$ gcc -Wall -Wextra t4_fixed.c -o t4f && ./t4f
inside main                : address of list[0] = 0x7fffa76a2650
inside addInterest : address of account = 0x7fffa76a2650
inside addInterest : address of account = 0x7fffa76a2658
inside addInterest : address of account = 0x7fffa76a2660
101 5200.00
102 12480.00
103 832.00
```

Analysis / Discussion

Task 4.1 – Locate the Defects

Defect 1 (balances do not change): addInterest(struct account a, ...) receives the structure **by value**. C makes a copy of list[i], the interest is added to the copy, and the copy is thrown away when the function ends. The original in main() is never touched.

Defect 2 (does not compile): p is a pointer (struct account *p = list), but the program uses p.number and p.balance. The dot operator works only on a structure variable. For a pointer the arrow operator is needed (p->number, p->balance). The compiler reported: "'p' is a pointer; did you mean to use '->'?"

Task 4.2 – Construct Evidence

The output above shows two **different addresses**: the account inside addInterest() is stored at a different address (inside the function's own stack frame) than list[0] in main(). So the function worked on a copy. The balances printed from main() are still 5000, 12000 and 800. In the repaired program the function prints the **same address** as list[0] for the first account, so it works on the original.

Task 4.3 – Repair the Program

addInterest() now takes **struct account *a** and updates a->balance. main() calls addInterest(&list;[i], 4). The result is 5200.00, 12480.00 and 832.00 (4% interest on 5000, 12000 and 800).

Task 4.4 – Prevent Recurrence (guideline)

Pass the structure	When to use
By value	Only for small structures that the function must not change, when a private copy is wanted (or when a modified copy is returned).

Pass the structure	When to use
By pointer	When the function must change the original structure, or when the structure is large (only one address is copied).
By const pointer	When the function only reads a large structure. It is fast like a pointer and the compiler stops accidental changes (for example print functions).

Alternative Concept and Complexity Analysis (pass by value and return)

```
struct account addInterest(struct account a, float rate) {
    a.balance = a.balance + a.balance * rate / 100;
    return a;
} ...
      list[i] = addInterest(list[i], 4);
```

This also works (output below is from t4_value.c, also run). Time: the structure is copied into the function and copied again on return, so 2 x sizeof(struct account) bytes per call. For struct account this is 2 x 8 = 16 bytes, while a pointer copies only one address (8 bytes). Stack space: by value uses sizeof(struct) for the parameter plus a temporary for the returned copy; by pointer uses a single 8-byte pointer. For a small structure the difference is tiny, but for a large one, like the 56-byte Patient in Task 1, by value costs 112 bytes per call against 8 bytes. In both cases the time grows only with the structure size and not with the number of accounts, but the pointer version is O(1) per call and the value version is O(sizeof(struct)).

```
$ gcc t4_value.c -o t4v && ./t4v
101 5200.00
102 12480.00
103 832.00
sizeof(struct account) = 8, sizeof(pointer) = 8
```

Result

Both defects were found and proved (different addresses; compiler message for p.number). The repaired program adds 4% interest to every account (5200.00, 12480.00, 832.00). I learned to pass a structure by pointer when the function must change it, and by const pointer when it only reads it.

TASK 5 — STORAGE CLASSES: THE SAME COUNTER FOUR WAYS (Hostel Mess Token)

Task / Problem Statement

Ahostel mess issues token numbers starting from 1001. The token function is called from more than one source file. Implement the token counter with (a) an automatic local variable, (b) a static local variable, (c) a global variable shared across two files with extern, and (d) a register loop variable; then change the requirement to two counters (veg and non-veg), compare the four storage classes and explain why the address of a register variable cannot be taken.

Aim

Tounderstand how scope and lifetime change with the storage class by implementing the same token counter in four ways.

Concept / Theory

auto variables arecreated when a block starts and destroyed when it ends. **static** local variables are created once and keep their value for the whole program, but are visible only in their function. A **global** variable is defined in one file and shared with other files by declaring it extern. A **register** variable is a request to keep a fast variable in a CPU register; its address cannot be taken.

Procedure / Algorithm

1. Write next_token() with a static local variable (starts at 1000, returns ++token).
2. Write auto_token() with an automatic variable for comparison.
3. Write a global counter in one file and use it in another file with extern; write a loop with a register variable.
4. Make two independent counters (veg and non-veg) for each version.
5. Run all programs, record the outputs and compare.

Program / Source Code

(a) Automatic local variable and (b) static local variable

```
#include <stdio.h>
int next_token(void) { static int token = 1000; return ++token; } int
auto_token(void) { int token = 1000; return ++token; }
int main(void)
{
    int s1 = next_token(), s2 = next_token(), s3 = next_token(); int a1 =
    auto_token(), a2 = auto_token(), a3 = auto_token(); printf("static: %d %d
    %d\n", s1, s2, s3);
    printf("auto       : %d %d %d\n", a1, a2, a3);
}   return 0;
```

(c) Global variable shared across two files with extern – file 1 (also has (d) register loop variable)

```
#include <stdio.h>
int         next_global(void);          /* defined in t5_counter.c */
extern int token;                       /* the global lives in t5_counter.c */
int main(void)
{
    int g1 = next_global();
    int g2 = next_global();
    int g3 = next_global();
    printf("global (extern):  %d  %d  %d\n",  g1,  g2,  g3);
    printf("token seen from main file = %d\n", token);

    register int i;                     /* register loop variable */
    printf("register loop          :");
    for (i = 1; i <= 3; i++) printf(" %d", 1000 + i);
}   printf("\n");
    return 0;
```

(c) file 2 – the global variable and its function

```
int token = 1000;                       /* global, shared across files */
int next_global(void) { return ++token; }
```

Task 5.2 – two independent counters (static version)

```
#include <stdio.h>
/* Two independent counters: veg and non-veg */
int next_veg(void)    { static int token = 1000; return ++token; }
int next_nonveg(void) { static int token = 2000; return ++token; }
int main(void)
{
    int v1 = next_veg(), v2 = next_veg();
    int n1 = next_nonveg(), n2 = next_nonveg();
    printf("veg    : %d %d\n", v1, v2); printf("nonveg : %d
    %d\n", n1, n2); return 0;
}
```

Alternative – counter kept in the caller and passed as a pointer

```
#include <stdio.h>
/* counter kept in the caller and passed as a pointer */ int
next_token(int *counter) { return ++(*counter); } int main(void)
{

    int veg = 1000, nonveg = 2000;
    int v1 = next_token(&veg), v2 = next_token(&veg);
    int n1 = next_token(&nonveg), n2 = next_token(&nonveg);
    printf("veg    : %d %d\n", v1, v2); printf("nonveg : %d
}   %d\n", n1, n2); return 0;
```

Order-of-evaluation experiment (three calls in one printf)

```
#include <stdio.h>
int next_token(void) { static int token = 1000; return ++token; } int
main(void)
{
    /* order of evaluation of arguments is NOT specified in C */
    printf("%d %d %d\n", next_token(), next_token(), next_token());
}   return 0;
```

Output Screenshot

```
$ gcc t5a.c -o t5a && ./t5a
static: 1001 1002 1003
auto : 1001 1001 1001
$ gcc t5_main.c t5_counter.c -o t5m && ./t5m global
(extern): 1001 1002 1003
token seen from main file = 1003
register loop        : 1001 1002 1003
$ gcc t5b.c -o t5b && ./t5b
veg    : 1001 1002
nonveg : 2001 2002
$ gcc t5d.c -o t5d && ./t5d
veg    : 1001 1002
nonveg : 2001 2002
$ gcc t5c.c -o t5c && ./t5c
1003 1002 1001
$ gcc t5_regaddr.c -o t5r 2>&1 | head -2
t5_regaddr.c: In function ‘main’:
t5_regaddr.c:4:5: error: address of register variable ‘r’ requested
```

Analysis / Discussion

Task 5.1 – Tokens produced

Version	Tokens for 3 calls	Reason
(a) Automatic local	1001 1001 1001	The variable is created and reset to 1000 at every call and destroyed at the end, so it always gives 1001.
(b) Static local	1001 1002 1003	The variable is created once and keeps its value between calls.
(c) Global with extern	1001 1002 1003	One variable is defined in file 2 and shared with file 1 using extern.

Version	Tokens for 3 calls	Reason
(d) Register loop variable	1001 1002 1003 (loop values 1000+i)	A register variable is only a fast loop counter; it is not a counter that survives a function call.

Task 5.2 – Change the Requirement (veg and non-veg counters)

Version	How much code changed
Automatic	Cannot work. Nothing is remembered between calls, so both counters would always print 1001.
Static	Small: one more function with its own static variable (about 2 lines changed/added). The counters cannot interfere, because each static variable is private to its function.
Global (extern)	Medium: a second global, a second function and one more extern line in the other file (about 4 lines in 2 files). Any function can change the globals by mistake.
Register	Not suitable. A register variable lives only inside one loop or function; two counters would need two loops.

Task 5.3 – Compare the Storage Classes

Storage class	Scope	Lifetime	Initial value	Risk of accidental modification
auto (local)	Inside the block	Until the block ends	Garbage if not set	Low (private), but the value is lost
static (local)	Inside the function	Whole program	0 if not set	Low (only that function can use it)
global + extern	All files	Whole program	0 if not set	High (every function and file can change it)
register	Inside the block	Until the block ends	Garbage if not set	Low (private)

Why the address of a register variable cannot be taken: a register variable may be kept in a CPU register, which has no memory address. So &r; is not allowed, and the compiler gave the error shown above ("address of register variable 'r' requested").

Order-of-evaluation experiment

When the three calls were moved into one printf, this machine printed **1003 1002 1001** (the arguments were evaluated from right to left), instead of 1001 1002 1003. C does not specify the order in which function arguments are evaluated, so the compiler can choose any order. Separate statements (int s1 = next_token(), s2 = ..., s3 = ...) give a fixed order, so they should be used when the order matters.

Alternative Concept and Complexity Analysis (pointer to a counter in the caller)

int next_token(int *counter) { return ++(*counter); } (program t5d.c, output above). **Time:** O(1) in every version, since each call does one increment. **Space and lifetime:** the counter is an ordinary variable in the caller (4 bytes), and it lives as long as the caller wants. The function itself has no hidden memory, so it is easy to test and has no global state. **Two independent counters:** very easy; just declare two variables (veg, nonveg) and pass the address of the one needed, with no change to the function at all. The only cost is that the caller must create and pass the counter.

Result

All four versions were implemented and the tokens were recorded: static and global gave 1001, 1002, 1003, and automatic gave 1001 each time. The static version was the easiest to change for two counters. I learned the scope and lifetime of each storage class and that the order of argument evaluation is not fixed in C.

TASK 6 — DESIGN CHALLENGE (College Canteen Inventory and Billing)

Task / Problem Statement

The college canteen sells packaged items (price and expiry date), cooked items (price and time prepared) and combo meals (made of up to three other items). It needs to record sales, update stock, warn about items expiring within two days and print the day's bill summary sorted by revenue. No design is given: interpret the problem, propose the data model, find the critical decisions, design the structures and functions, and implement and demonstrate the program.

Aim

Todesign and build a complete small program using nested structures, an array of structures, a tagged union, structure pointers and static data.

Concept / Theory

A**tagged union**stores one of several different details in the same memory, with an enum tag to say which one is valid (expiry date, prepared time or combo parts). **Nested structures** (Date and Time) keep related values together. **Structure pointers** let the functions change the real item and avoid copying. A **static** variable at file level keeps one private copy for the whole program.

Procedure / Algorithm

1. Listthe entities,their data and the rules, and write the rules as testable statements.
2. Define Date, Time, enum Kind and struct Item with a union, and an array of items.
3. Write record_sale() (check stock first), expiry_warning(), sort_by_revenue() and print_summary() using structure pointers.
4. In main(), do normal sales, a sale that exceeds stock, an expiry check and print the sorted summary.
5. Run the program and compare the output with the requirements.

Program / Source Code

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 10

struct Date { int day, month, year; };
struct Time { int hour, minute; };

enum Kind { PACKAGED, COOKED, COMBO };

struct Item {
      int id;
      char   name[20];
      enum   Kind   kind;
      float   price;    int
      stock;
      int sold; union {                    /* number sold today */
                                           /* tagged union: depends on kind */
            struct Date expiry;            /* PACKAGED */
            struct Time prepared;          /* COOKED    */
            int parts[3];                  /* COMBO: ids of up to three items */
      } info;
};

static  struct  Date  today  =  {13,  11,  2026};  static  /* static: today's date */
float day_total = 0;                                        /* static: running total */

/* Converts a date to a day number so two dates can be subtracted.
    (Simple version: ignores the 100/400-year leap rules.) */
int days_of(const struct Date *d) {

      static const int md[] = {31,28,31,30,31,30,31,31,30,31,30,31};
      int days = d->year * 365 + (d->year - 1) / 4;
}     for (int m = 0; m < d->month - 1; m++) days += md[m];
      return days + d->day;
```

```
int find(const struct Item list[], int n, int id)
{
    for (int i = 0; i < n; i++) if (list[i].id == id) return i;
}   return -1;

/* Record a sale safely: refuse if stock is not enough */ int
record_sale(struct Item list[], int n, int id, int qty) {


    int i = find(list, n, id);
    if (i < 0) { printf("Item %d not found\n", id); return 0; }
    struct Item *p = &list[i];
    if (qty > p->stock) {
        printf("Sale refused: %s needs %d but only %d in stock\n", p->name, qty, p->stock);
        return 0;
    }
    p->stock -= qty;
    p->sold += qty;
}   day_total += qty * p->price;
    printf("Sold %d x %s = Rs %.2f\n", qty, p->name, qty * p->price); return 1;
void expiry_warning(const struct Item list[], int n)
{




    for (int i = 0; i < n; i++) {
        if (list[i].kind != PACKAGED) continue;
        int left = days_of(&list[i].info.expiry) - days_of(&today);
}       if (left <= 2)
            printf("WARNING: %s %s (%d day(s) left)\n", list[i].name,
void sort_by_revenue(struct Item list[], int n)
                   left < 0 ? "has expired" : "expires soon", left);
{   }




    for (int i = 0; i < n - 1; i++) {
}       int max = i;
        for (int j = i + 1; j < n; j++)
void print_summary(const struct Item list[], int n)
            if (list[j].sold * list[j].price > list[max].sold * list[max].price) max = j;
{       if (max != i) { struct Item t = list[i]; list[i] = list[max]; list[max] = t; }
    }

}

int main(void)
{   printf("%-12s %5s %8s\n", "Item", "Sold", "Revenue");
    for (int i = 0; i < n; i++)
        printf("%-12s %5d %8.2f\n", list[i].name, list[i].sold, list[i].sold * list[i].price);
    printf("%-12s %5s %8.2f\n", "TOTAL", "", day_total);




    struct Item menu[MAX_ITEMS] = {
        {1, "Biscuits", PACKAGED, 10, 20, 0, {.expiry = {14, 11, 2026}}},
        {2, "Juice",    PACKAGED, 25, 15, 0, {.expiry = {30, 12, 2026}}},
        {3, "Samosa",   COOKED,   12, 30, 0, {.prepared = {9, 30}}},
        {4, "Combo",    COMBO,    40, 10, 0, {.parts = {2, 3, 0}}}
    };
    int n = 4;

    printf("--- Normal sales ---\n");
    record_sale(menu, n, 3, 5);
    record_sale(menu, n, 2, 4);
}   record_sale(menu, n, 4, 2);
    printf("--- Sale that exceeds stock ---\n");
    record_sale(menu, n, 1, 50);
    printf("--- Expiry check (today = %02d/%02d/%d) ---\n", today.day, today.month, today.year);
    expiry_warning(menu, n);
    printf("--- Bill summary sorted by revenue ---\n");
    sort_by_revenue(menu, n);
    print_summary(menu, n);
    return 0;
```

Output Screenshot

```
$ gcc -Wall -Wextra t6.c -o t6
$ ./t6
--- Normal sales ---
Sold 5 x Samosa = Rs 60.00
Sold 4 x Juice = Rs 100.00
Sold 2 x Combo = Rs 80.00
--- Sale that exceeds stock ---
Sale refused: Biscuits needs 50 but only 20 in stock --- Expiry
check (today = 13/11/2026) ---
WARNING: Biscuits expires soon (1 day(s) left)
--- Bill summary sorted by revenue ---
Item   Juice          Sold  Revenue
Combo                   4   100.00
Samosa                  2    80.00
Biscuits                5    60.00
TOTAL                   0     0.00
                            240.00
```

Analysis / Discussion

Stage 1 – Interpret the Problem (entities, data, rules)

Entity	Data
Item (packaged / cooked / combo)	id, name, kind, price, stock, quantity sold today, and one extra detail that depends on the kind
Packaged item	expiry date (Date: day, month, year)
Cooked item	time prepared (Time: hour, minute)
Combo meal	up to three other item ids
Bill summary	item name, quantity sold, revenue; day total

Requirements as testable statements:

R1. Selling q items (q <= stock) reduces the stock by q and adds q x price to the day's total.

R2. Selling more than the stock is refused, a message is printed and the stock does not change.

R3. A packaged item that expires within 2 days (or has expired) gets a warning; cooked items and combos are never checked.

R4. The bill summary is printed in descending order of revenue (sold x price), with a total.

Stage 2 – Propose the Data Model

Nested structures: Date and Time are small structures placed inside the item. **Array of structures:** struct Item menu[10] holds all items. **Tagged union:** the field kind (enum PACKAGED, COOKED, COMBO) is the tag, and a union holds either the expiry Date, the prepared Time or the three combo ids, because an item has only one of them. This saves memory and keeps each item as one record.

Stage 3 – Critical Design Decisions

Safe stock update: record_sale() first finds the item, checks qty <= stock, and only then changes stock and sold. A refused sale changes nothing. **Expiry check:** both dates are converted to day numbers by days_of(struct Date *) and subtracted; a result of 2 or less (or negative) gives a warning. The union member is read only when kind == PACKAGED. **Static or global:** today's date and the day's total are file-level static variables (one copy, private to the file, live for the whole program). The menu is not global; it is passed to the functions by pointer.

Stage 4 – Structures, Function Prototypes and Pseudocode

```
struct Date { int day, month, year; };                  struct Time { int hour, minute; };
enum Kind { PACKAGED, COOKED, COMBO };
struct Item { int id; char name[20]; enum Kind kind; float price; int stock, sold;
                union { struct Date expiry; struct Time prepared; int parts[3]; } info; };

int days_of(const struct Date *d);
int find(const struct Item list[], int n, int id);
int record_sale(struct Item list[], int n, int id, int qty);
void expiry_warning(const struct Item list[], int n);
void sort_by_revenue(struct Item list[], int n);
void print_summary(const struct Item list[], int n);

Record a sale:                                  Expiry warning:
 i = find(item id)                               for each item
 if i not found: print error, stop               if kind is not PACKAGED: skip
```

```
p=   pointer to item      i
ifqty > p->stock:
              print 'sale refused', stop
p->stock = p->stock     -    qty
p->sold = p->sold       +  qty
day_total = day_total       +  qty   *p->price
```

```
left    = days_of(expiry) - days_of(today)
if left<= 2:
       print    warning (expired if left < 0)
```

Stage 5 – Implement and Demonstrate

The program above demonstrates: normal sales (Samosa, Juice, Combo), a sale that exceeds stock (50 biscuits, only 20 available, so it was refused), an expiring item (Biscuits expire tomorrow, 1 day left) and the summary sorted by revenue (Juice 100, Combo 80, Samosa 60, Biscuits 0, total 240). The combo sale does not reduce the stock of its parts, and days_of() ignores the 100/400-year leap rules; these are simplifications.

Alternative Concept and Complexity Analysis (linked list)

If the items were kept in a linked list of dynamically allocated structures (each node holding an item and a next pointer): **add** at the head is O(1) (an array adds at the end in O(1) while there is room, but has a fixed maximum, 10 here). **Remove** is O(n) to find the node and then O(1) to unlink it, while the array needs O(n) to find it and O(n) to shift the elements. **Search** by id is O(n) in both. The array allows fast access by index and is stored in one block, which is good for sorting; the list has no fixed limit and removal does not shift data. **Extra space:** each node needs one next pointer (8 bytes on a 64-bit system), plus the memory manager's bookkeeping, so 10 items need at least 80 extra bytes. Sorting a list by swapping nodes is more difficult. For a small canteen menu (10 items) the array of structures is simpler and good enough.

Result

The canteen program works for all required cases: normal sales updated the stock, the oversized sale was refused without changing the stock, the expiring item was warned, and the summary was sorted by revenue with the correct total (Rs 240). I learned how nested structures, a tagged union and structure pointers work together in one design.

Test Cases / Validation Table

Differentinputs,theexpectedresults,theactualresults from running the programs, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 1: add patients aged 45, 28, 62, then sort by age	Ages in order: 28 45 62	28 45 62	Pass
2	Task 2: parcels 501 (4.5 kg), 502 (1.2 kg), 503 (2.8 kg) sorted by weight	Ids in order: 502 503 501	502 503 501	Pass
3	Task 3: sizeof union and struct reading	8 and 12	8 and 12	Pass
4	Task 4: 4% interest on 5000, 12000, 800	5200.00 12480.00 832.00	5200.00 12480.00 832.00	Pass
5a	Task 5: static token, 3 calls	1001 1002 1003	1001 1002 1003	Pass
5b	Task 5: automatic token, 3 calls	1001 1001 1001	1001 1001 1001	Pass
6a	Task 6: sell 50 Biscuits when stock is 20	Sale refused, nothing sold	Sale refused, nothing sold	Pass
6b	Task 6: Biscuits expire 14/11/2026, today 13/11/2026	Warning, 1 day left	Warning, 1 day left	Pass

Final Submission Checklist

[ ] Studentandcourse detailscompleted

[ ] Task statement, aim, theory, and algorithm included

[ ] Program code is readable and runs successfully

[ ] Output screenshot included

[ ] Analysis and result written in own words

[ ] Test cases completed where applicable

[ ] GitHub repository link added if required